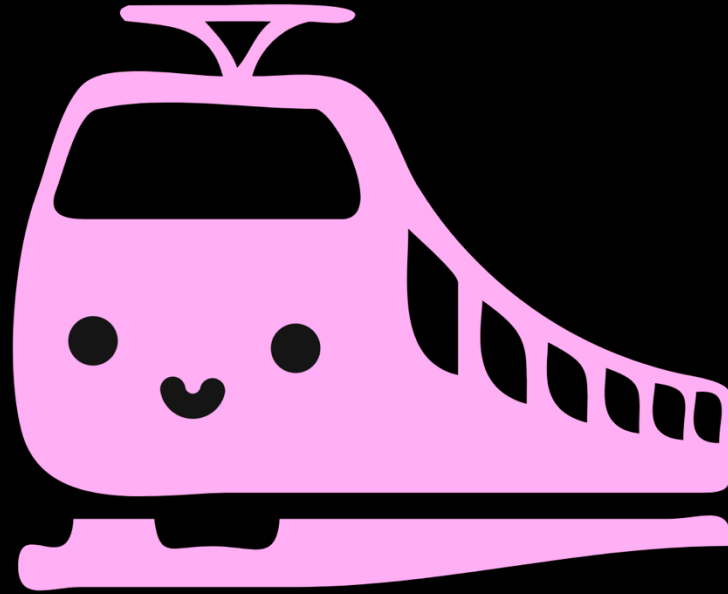


Riding the Sour Train: Building surtoget.no with Glean





- Developer/tech lead/architect
- Worked with startups, healthcare, insurance, banking, transportation and stuff.
- Bad at FE

This talk



This talk





[Hjem](#) [Nyheter](#) [Om Surtoget](#) [Ofte stilte spørsmål](#)



Velkommen til Surtoget! Kjenner du på den spesielle blandingen av håp og fortvilelse hver gang du setter deg på Sørlandsbanen?

Her deler vi historier fra virkeligheten, belyser problemene og gir deg verktøyene du trenger for å takle en hverdag med en av Norges mest utilregnelige toglinjer.

● Akkurat nå: Sørlandsbanen er forsinket... Igjen 😞

STATISTIKK

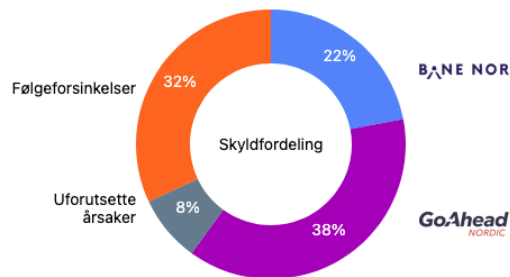
Siste rapport (August)

[Dette året så langt](#)

[Punktlighet over tid](#)

25.6%

sjanse for forsinkelse



PENGER TILBAKE VED FORSINKELSE

Sur fordi toget er forsinket? Slik får du penger tilbake!

Vi vet, det er kjipt å være forsinket. Men visste du at du kan ha krav på refusjon? Her er en enkel oversikt over når du kan juble hele veien til banken, og når du må bite i det sure eplet.

🟢 Dette har du krav på

Er du mer enn 60 minutter forsinket med Sørtoget? Da får du 50% av billett-kostnaden tilbake. Enkelt og greit!

🔴 ...og dette har du ikke krav på

Dessverre, hvis forsinkelsen skyldes ekstremvær, naturkatastrofer, eller streik, må du nok bare smøre deg med tålmodighet (og en god bok).



Hjem **Nyheter** Om Surtoget Ofte stilte spørsmål



Velkommen til Surtoget! Kjenner du på den spesielle blandingen av håp og fortvilelse hver gang du setter deg på Sørlandsbanen?

Her deler vi historier fra virkeligheten, belyser problemene og gir deg verktøyene du trenger for å takle en hverdag med en av Norges mest utilregnelige toglinjer.

● Akkurat nå: Sørlandsbanen er forsinket... Igjen 😞

STATISTIKK

Siste rapport (August)

Dette året så langt

Punktlighet over tid

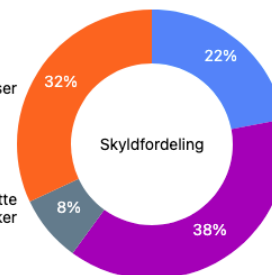
ENTUR

25.6%

sjanse for forsinkelse

Følgforsinkelser

Uforutsette årsaker



BANE NOR

GoAhead
NORDIC

PENGER TILBAKE VED FORSINKELSE

Sur fordi toget er forsinket? Slik får du penger tilbake!

Vi vet, det er kjipt å være forsinket. Men visste du at du kan ha krav på refusjon? Her er en enkel oversikt over når du kan juble hele veien til banken, og når du må bite i det sure eplet.

🟢 Dette har du krav på

Er du mer enn 60 minutter forsinket med Sørtoget? Da får du 50% av billett-kostnaden tilbake. Enkelt og greit!

🔴 ...og dette har du ikke krav på

Dessverre, hvis forsinkelsen skyldes ekstremvær, naturkatastrofer, eller streik, må du nok bare smøre deg med tålmodighet (og en god bok).

But what is a Sørlandsbanen anyway?



But what is a Sørlandsbanen anyway?



Type 73 (20)



But what is a Sørlandsbanen anyway?



But what is a Sørlandsbanen anyway?



But what is a Sørlandsbanen anyway?



But what is a Sørlandsbanen anyway?



But what is a Sørlandsbanen anyway?



But what is a Sørlandsbanen anyway?



But what is a Sørlandsbanen anyway?



But what is a Sørlandsbanen anyway?





[Hjem](#) [Nyheter](#) [Om Surtoget](#) [Ofte stilte spørsmål](#)



Velkommen til Surtoget! Kjenner du på den spesielle blandingen av håp og fortvilelse hver gang du setter deg på Sørlandsbanen?

Her deler vi historier fra virkeligheten, belyser problemene og gir deg verktøyene du trenger for å takle en hverdag med en av Norges mest utilregnelige toglinjer.

● Akkurat nå: Sørlandsbanen er forsinket... Igjen 😞

STATISTIKK

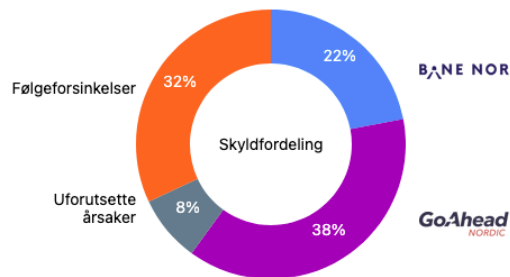
Siste rapport (August)

[Dette året så langt](#)

[Punktlighet over tid](#)

25.6%

sjanse for forsinkelse



PENGER TILBAKE VED FORSINKELSE

Sur fordi toget er forsinket? Slik får du penger tilbake!

Vi vet, det er kjipt å være forsinket. Men visste du at du kan ha krav på refusjon? Her er en enkel oversikt over når du kan juble hele veien til banken, og når du må bite i det sure eplet.

🟢 Dette har du krav på

Er du mer enn 60 minutter forsinket med Sørtoget? Da får du 50% av billett-kostnaden tilbake. Enkelt og greit!

🔴 ...og dette har du ikke krav på

Dessverre, hvis forsinkelsen skyldes ekstremvær, naturkatastrofer, eller streik, må du nok bare smøre deg med tålmodighet (og en god bok).



[Hjem](#) [Nyheter](#) [Om Surtoget](#) [Ofte stilte spørsmål](#)



Velkommen til Surtoget! Kjenner du på den spesielle blandingen av håp og fortvilelse hver gang du setter deg på Sørlandsbanen?

Her deler vi historier fra virkeligheten, belyser problemene og gir deg verktøyene du trenger for å takle en hverdag med en av Norges mest utilregnelige toglinjer.

● Akkurat nå: Sørlandsbanen er forsinket... Igjen 😞

Go Ahead

NORDIC

PENGER TILBAKE VED FORSINKELSE

Sur fordi toget er forsinket? Slik får du penger tilbake!

Vi vet, det er kjipt å være forsinket. Men visste du at du kan ha krav på refusjon? Her er en enkel oversikt over når du kan juble hele veien til banken, og når du må bite i det sure eplet.

🟢 Dette har du krav på

Er du mer enn 60 minutter forsinket med Sørtoget? Da får du 50% av billett-kostnaden tilbake. Enkelt og greit!

🔴 ...og dette har du ikke krav på

Dessverre, hvis forsinkelsen skyldes ekstremvær, naturkatastrofer, eller streik, må du nok bare smøre deg med tålmodighet (og en god bok).

togene hjelper ikke

John Mikael Lindbakk fra Lunde i Telemark, som står bak nettsiden surtoget.no, sier han ikke forventer store forbedringer, hvis Vy får overta Sørlandsbanen.



Akk

og fortvilelse hver gang du

verktøyene du trenger for å takle

FOTO: Bø blad

at toga skulle gå kvar halvannan time, har forseinkingane blitt meir merkbare. Når eitt tog er forseinka, spreier problemet seg raskt til neste.

Vegen vidare

Noreg må no svare ESA innom to månader. Viss overvakingorganet ikkje godtek forklaringa, kan saka ende i EFTA-domstolen.

Eit mogleg utfall er at trakten må lysast ut på trosser at Vy mistar retten til å reise Sørlandsbanen på vilkår.

– Eg trur ikkje det vil gjere saka noko betre. Det er eldre togsett og linjekapasiteten som er for dårleg. Eg har lita tru på at byte av logo og fargar på togsetta vil gjere nokon skilnad, seier han.

Likevel håpar Lindbakk på ein positiv bieffekt: – Det ein kan håpe på, er at Vy er meir villig til å gjenopprette nokre av avgangane som blei fjerna i Telemark.

Tal frå Bane Nor viser at punktlegheta på Sørlandsbanen har gått ned dei siste ti åra. Etter at politikaane i 2015 bestemte

na veg ut, og det kan det bli bråk av.

manfatting kan ein seie at oplysningen på høgresida meiner Regjeringa medvite utfordrar EØS-avtalen og risikerer eit juridisk nederlag. Regjeringa og venstresida hevdar at Noreg må ha rett til å styre sin eigen jernbane utan overstyring frå Brussel.

– Trur ikkje det blir betre

John Mikael Lindbakk frå Lunde i Telemark, som står bak nettsida surtoget.no, seier han ikkje ventar stor framgang sjølv om Vy får overta Sørlandsbanen.

med Sørtoget? Da får
Enkelt og greit!



togene hjelper ikke

John Mikael Lindbakk fra Lunde i Telemark, som står bak nettsiden surtoget.no, sier han ikke forventer



og fortvilelse hver gang du

verktøyene du trenger for å takle

Bergen

Oslo

Drammen

Kongsberg

Bø

Stavanger

Nelaug

Egersund

Flekkefjord

Vennesla

Kristiansand

L
for
nasj

ne. Skapte
ikk nok av
oppmerksom

med Sørtoget? Da får
. Enkelt og greit!

nettsida
ikkje ventar stor framg
om Vy får overta Sørlandsba
nen.

at politika

: Bø blad
ar halvann-
ingane blitt
r eitt tog er
problemet seg

vare ESA inn
iss overvakin
godtek forkla
ende i EFTA-d

leg utfall er a
å lysast ut på
mistar retten
rlandsbanen p

togene hjelper ikke

John Mikael Lindbakk fra Lunde i Telemark, som står bak nettsiden surtoget.no, sier han ikke forventer



og fortvilelse hver gang du

verktøyene du trenger for å takle

Bergen

Oslo



Egersund

Flekkefjord

Kristiansand

Nelaug

med Sørtoget? Da får
. Enkelt og greit!

nettsida
ikkje ventar stor framg
om Vy får overta Sørlandsba
nen.

at politika

togene hjelper ikke

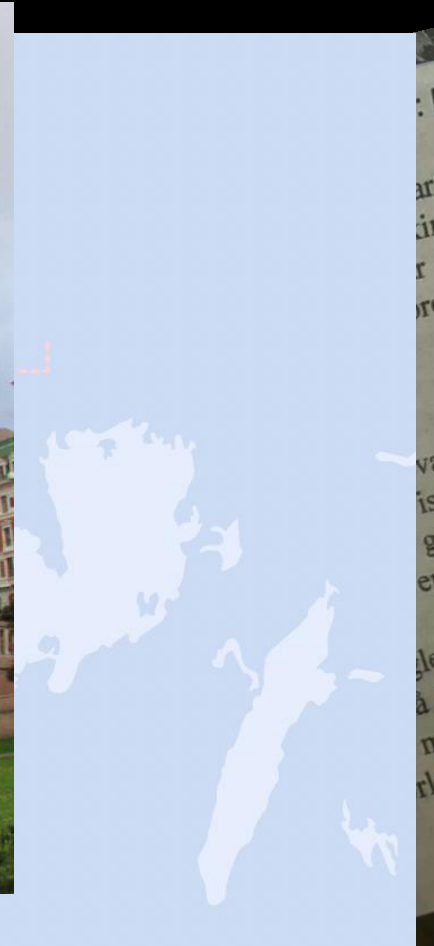
John Mikael Lindbakk fra Lunde i

Telemark, som står bak nettsiden



surtoget no, sier han ikke forventer

og fortvilelse hver gang du



L
for
nasj

for John Mikael Lindbakk har togturene mellom Oslo og Lunde blitt mer frustrerende enn glede. Derfor tok han saken i hendene og lanserte surtoget som et opprop for å få Sørlandsbanen igjen.
ne. Skapte nok av oppmerksomhet.

med Sørtoget? Da får
Enkelt og greit!

nettsida
ikkje ventar stor framgang
om Vy får overta Sørlandsbanen.

at politikk

: Bø blad
ar halvann-
ingane blitt
r eitt tog er
problemet seg

vare ESA inn
iss overvakin
godtek forkla
ende i EFTA-d

leg utfall er a
å lysast ut på
mistar retten t
rlandsbanen p

togene hjelper ikke

John Mikael Lindbakk fra Lunde i

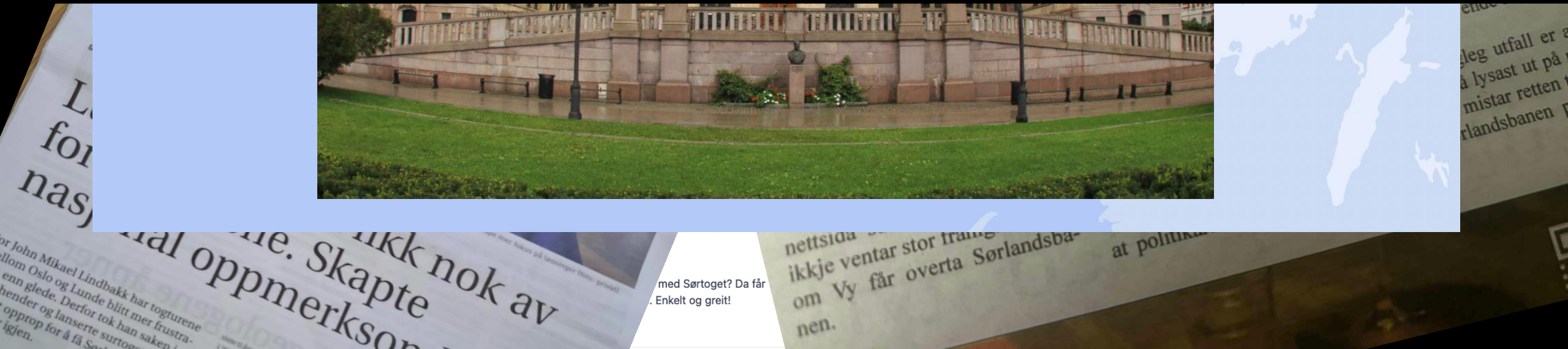
Telemark, som står bak nettsiden



surtoget no, sier han ikke forventer

og fortvilelse hver gang du

Developers are uniquely equipped to turn their craft into
social impact







YouTube^{NO}

Search



+ Create



```
new("wibble")  
|> with_icon(Sparkles)
```

```
new("wobble")  
|> with_colour(Error)  
|> is_disabled(True)
```

```
new("woo")  
|> with_icon(Confetti)  
|> with_style(Outline)  
|> with_colour(Success)  
|> with_icon(Sparkles)
```



Phantom Types and the Builder Pattern in Gleam • Hayleigh Thompson FOSDEM '24

All

From Hayleigh Thompson

Computer Science

Link: <https://www.youtube.com/watch?v=3lYHFctx2Ks>


```
1 pub fn send_welcome_email(email: String) -> Result(String, String) {  
2     // Send email to user...  
3     Ok("Welcome email sent to " <> email)  
4 }
```



```
1 pub fn send_welcome_email(email: String) -> Result(String, String) {
2     use _ <- result.try(is_valid_email(email))
3     use is_user_registered <- result.try(is_user_registered(email))
4     // Other checks?
5
6     case is_user_registered {
7         True -> {
8             // Send email to user
9             Ok("Welcome email sent to " <> email)
10        }
11        False -> Error("User not registered in the system")
12    }
13 }
```



```
1 pub fn send_welcome_email(email: String) -> Result(String, String) {
2     use _ <- result.try(is_valid_email(email))
3     use is_user_registered <- result.try(is_user_registered(email))
4     // Other checks?
5
6     case is_user_registered {
7         True -> {
8             // Send email to user
9             Ok("Welcome email sent to " <> email)
10        }
11        False -> Error("User not registered in the system")
12    }
13 }
```

Dependencies and tight coupling




```
1 pub fn main() -> Nil {
2     // Ok
3     let _ = welcome_sender.send_welcome_email("wibbly@wobbly.wabbely")
4     // Still Ok, but will fail. Why should we even allow this?
5     let _ = welcome_sender.send_welcome_email("wibbly")
6     //... Code
```

```
1 pub fn send_welcome_email(email: String) -> Result(String, String) {
2     use _ <- result.try(is_valid_email(email))
3     use is_user_registered <- result.try(is_user_registered(email))
4     // Other checks?
5
6     case is_user_registered {
7         True -> {
8             // Send email to user
9             Ok("Welcome email sent to " <> email)
10        }
11        False -> Error("User not registered in the system")
12    }
13 }
```



```
1 pub opaque type ValidEmail {
2   ValidEmail(value: String)
3 }
4
5 pub fn new(address: String) -> Result(ValidEmail, String) {
6   use _ <- result.try(is_valid_email(address))
7   use is_user_registered <- result.try(is_user_registered(address))
8   // Other checks?
9   case is_user_registered {
10     True -> Ok(ValidEmail(address))
11     False -> Error("User not registered in the system")
12   }
13 }
14
15 pub fn send_welcome_email(email: ValidEmail) -> Result(String, String) {
16   // Send email to user
17   Ok("Welcome email sent to " <> email.value)
18 }
```

error: Type mismatch

```
  /Users/lindbakk/repos/presentations/gleam-gathering-2026/demo/src/demo.gleam:8:45
8 |   let _ = welcome_sender.send_welcome_email("wibbly")
                                     ^^^^^^^^^
```

Expected type:

`welcome_sender.ValidEmail`

Found type:

`String`

```
1 pub fn main() -> Result(Nil, String) {
2     // Ok
3     use good_email_result <- result.try(welcome_sender.new(
4         "wibbly@wobbly.wabbely",
5     ))
6     let _ = welcome_sender.send_welcome_email(good_email_result)
7
8     use bad_email_result <- result.try(welcome_sender.new("wibbly"))
9
10    // Code will never reach since "wibbly" is not a valid email
11    let _ = welcome_sender.send_welcome_email(bad_email_result)
12    //... Code
```

```
1 pub fn main() -> Result(Nil, String) {
2     // Ok
3     use good_email_result <- result.try(welcome_sender.new(
4         "wibbly@wobbly.wabbely",
5     ))
6     let _ = welcome_sender.send_welcome_email(good_email_result)
7
8     use bad_email_result <- result.try(welcome_sender.new("wibbly"))
9
10    // Code will never reach since "wibbly" is not a valid email
11    let _ = welcome_sender.send_welcome_email(bad_email_result)
12    //... Code
```



```
1 pub opaque type ValidEmail {
2   ValidEmail(value: String)
3 }
4
5 pub fn new(address: String) -> Result(ValidEmail, String) {
6   use _ <- result.try(is_valid_email(address))
7   use is_user_registered <- result.try(is_user_registered(address))
8   // Other checks?
9   case is_user_registered {
10     True -> Ok(ValidEmail(address))
11     False -> Error("User not registered in the system")
12   }
13 }
14
15 pub fn send_welcome_email(email: ValidEmail) -> Result(String, String) {
16   // Send email to user
17   Ok("Welcome email sent to " <> email.value)
18 }
```

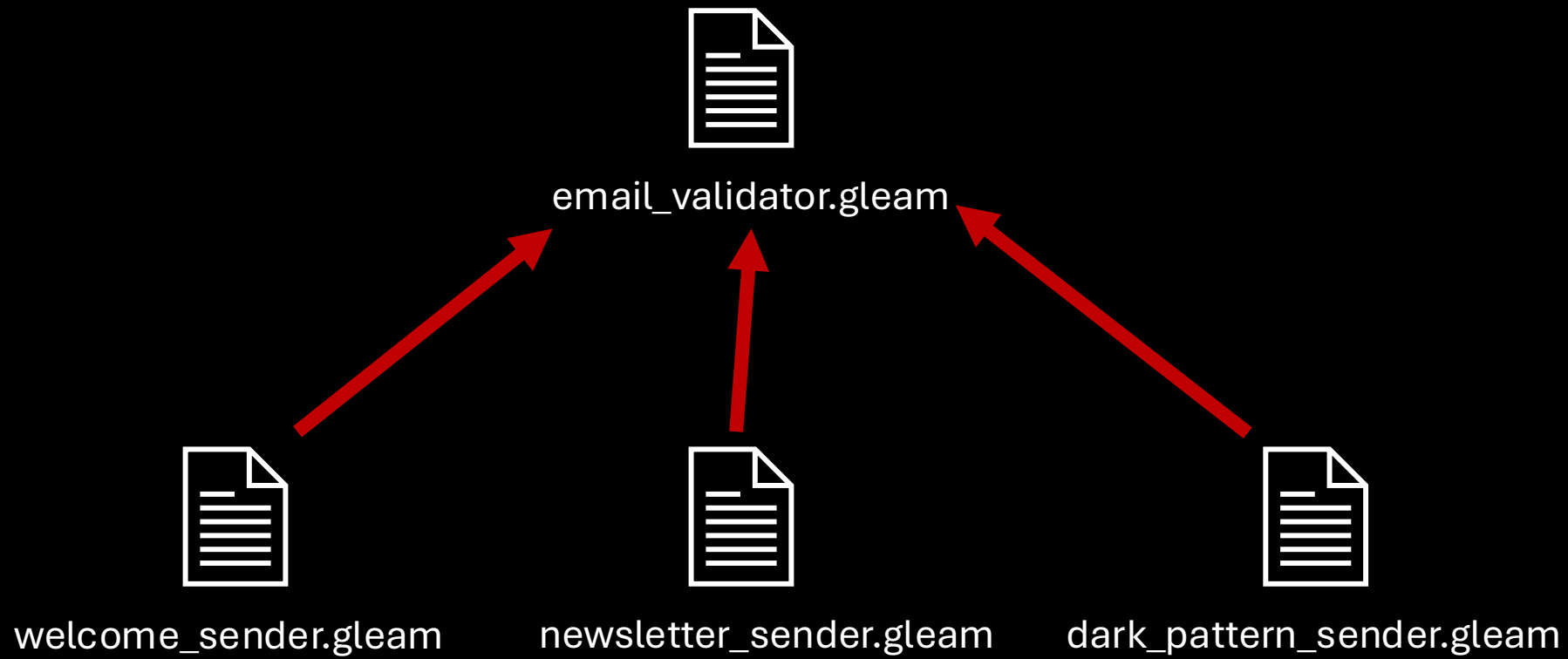


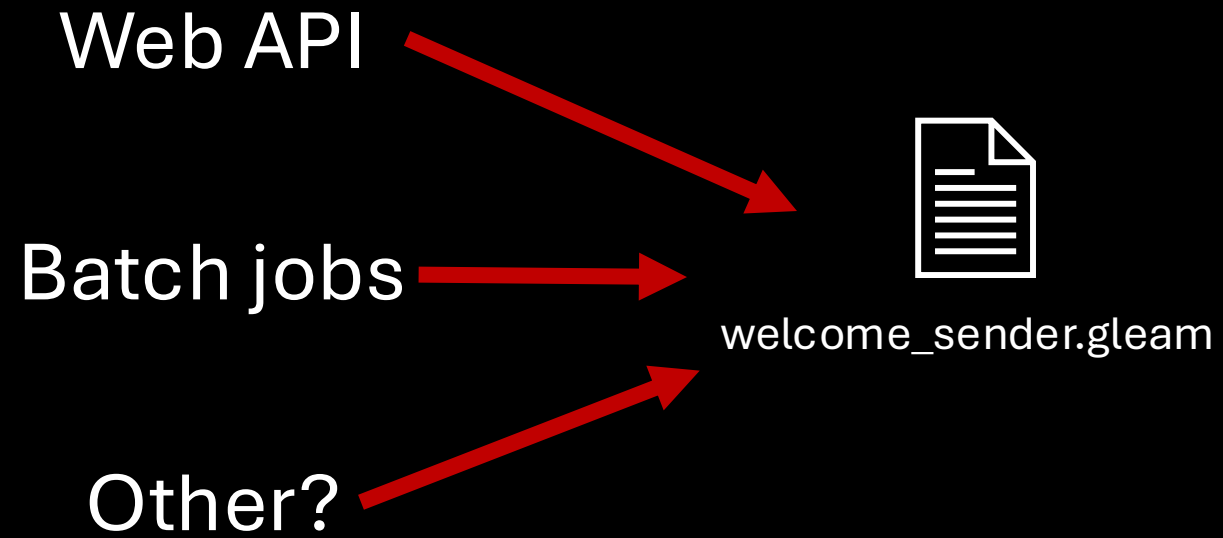
email_validator.gleam



welcome_sender.gleam

```
1 pub opaque type ValidEmail {
2   ValidEmail(value: String)
3 }
4
5 pub fn new(address: String) -> Result(ValidEmail, String) {
6   use _ <- result.try(is_valid_email(address))
7   use is_user_registered <- result.try(is_user_registered(address))
8   // Other checks?
9   case is_user_registered {
10     True -> Ok(ValidEmail(address))
11     False -> Error("User not registered in the system")
12   }
13 }
14
15 pub fn send_welcome_email(email: ValidEmail) -> Result(String, String) {
16   // Send email to user
17   Ok("Welcome email sent to " <> email.value)
18 }
```





All forced to validate!

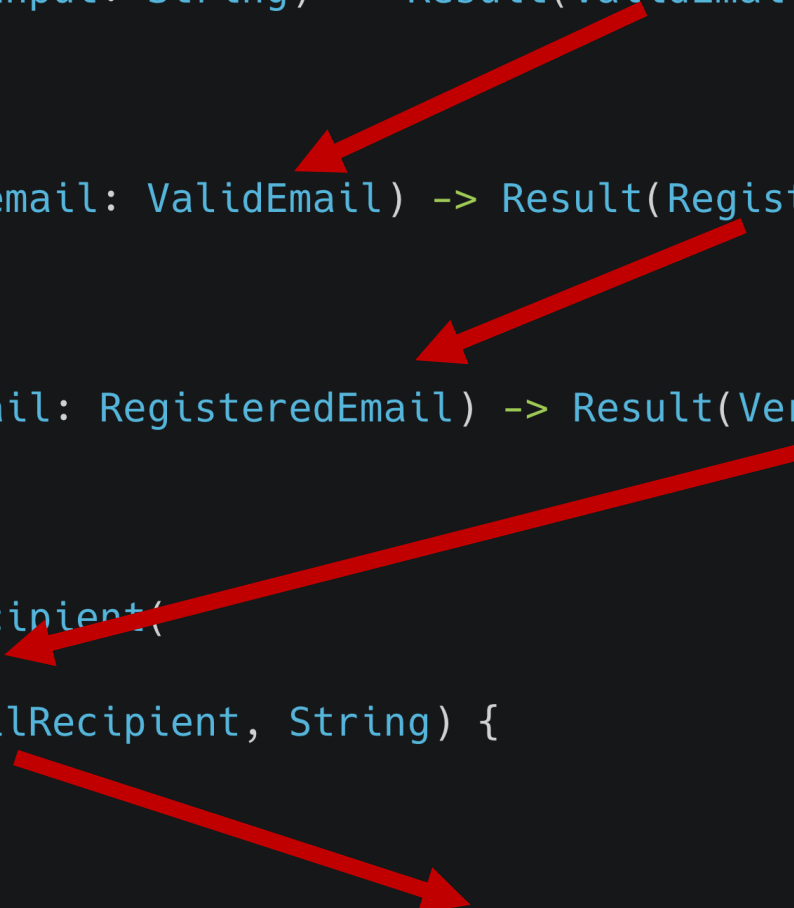

```
1 pub opaque type ValidEmail {  
2     ValidEmail(value: String)  
3 }  
4  
5 pub opaque type RegisteredEmail {  
6     RegisteredEmail(value: String)  
7 }  
8  
9 pub opaque type VerifiedEmail {  
10     VerifiedEmail(value: String)  
11 }  
12  
13 pub opaque type WelcomeEmailRecipient {  
14     WelcomeEmailRecipient(value: String)  
15 }
```

```
1 pub opaque type ValidEmail {  
2   ValidEmail(value: String)  
3 }  
4  
5 pub opaque type RegisteredEmail {  
6   RegisteredEmail(value: String)  
7 }  
8  
9 pub opaque type VerifiedEmail {  
10  VerifiedEmail(value: String)  
11 }  
12  
13 pub opaque type WelcomeEmailRecipient {  
14  WelcomeEmailRecipient(value: String)  
15 }
```

```
1 pub fn validate_email(input: String) -> Result(ValidEmail, String) {  
2   // Code...  
3 }  
4  
5 pub fn register_email(email: ValidEmail) -> Result(RegisteredEmail, String) {  
6   // Code...  
7 }  
8  
9 pub fn verify_email(email: RegisteredEmail) -> Result(VerifiedEmail, String) {  
10  // Code...  
11 }  
12  
13 pub fn make_welcome_recipient(  
14   email: VerifiedEmail,  
15 ) -> Result(WelcomeEmailRecipient, String) {  
16   // Code...  
17 }  
18  
19 pub fn send_welcome_email(_recipient: WelcomeEmailRecipient) -> Nil {  
20   // Code...  
21 }
```

```
1 pub opaque type ValidEmail {  
2   ValidEmail(value: String)  
3 }  
4  
5 pub opaque type RegisteredEmail {  
6   RegisteredEmail(value: String)  
7 }  
8  
9 pub opaque type VerifiedEmail {  
10   VerifiedEmail(value: String)  
11 }  
12  
13 pub opaque type WelcomeEmailRecipient {  
14   WelcomeEmailRecipient(value: String)  
15 }
```

```
1 pub fn validate_email(input: String) -> Result(ValidEmail, String) {  
2   // Code...  
3 }  
4  
5 pub fn register_email(email: ValidEmail) -> Result(RegisteredEmail, String) {  
6   // Code...  
7 }  
8  
9 pub fn verify_email(email: RegisteredEmail) -> Result(VerifiedEmail, String) {  
10  // Code...  
11 }  
12  
13 pub fn make_welcome_recipient(  
14   email: VerifiedEmail,  
15 ) -> Result>WelcomeEmailRecipient, String) {  
16   // Code...  
17 }  
18  
19 pub fn send_welcome_email(_recipient: WelcomeEmailRecipient) -> Nil {  
20   // Code...  
21 }
```



```
1 pub opaque type ValidEmail {  
2   ValidEmail(value: String)  
3 }  
4  
5 pub opaque type RegisteredEmail {  
6   RegisteredEmail(value: String)  
7 }  
8  
9 pub opaque type VerifiedEmail {  
10  VerifiedEmail(value: String)  
11 }  
12  
13 pub opaque type WelcomeEmailRecipient {  
14  WelcomeEmailRecipient(value: String)  
15 }
```

```
1 pub fn validate_email(input: String) -> Result(ValidEmail, String) {  
2   // Code...  
3 }  
4  
5 pub fn register_email(email: ValidEmail) -> Result(RegisteredEmail, String) {  
6   // Code...  
7 }  
8  
9 pub fn verify_email(email: RegisteredEmail) -> Result(VerifiedEmail, String) {  
10  // Code...  
11 }  
12  
13 pub fn make_welcome_recipient(  
14   email: VerifiedEmail,  
15 ) -> Result(WelcomeEmailRecipient, String) {  
16   // Code...  
17 }  
18  
19 pub fn send_welcome_email(_recipient: WelcomeEmailRecipient) -> Nil {  
20   // Code...  
21 }
```

```
1 pub fn onboard_and_welcome(input: String) -> Result(Nil, String) {  
2   input  
3   |> validate_email  
4   |> result.try(register_email)  
5   |> result.try(verify_email)  
6   |> result.try(make_welcome_recipient)  
7   |> result.map(send_welcome_email)  
8 }
```


“Make Illegal States Unrepresentable”

- Yaron Minsky

What I want you to remember:

Use your technical skills to amplify your voice (for good)

Use types to make illegal states a compile-time issue

